

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 06 · LECTURE NOTES

Graph Search and Decomposition

Two traversals, and a surprising amount of structure.

MODULE OUTCOMES

Choose between adjacency list and matrix on the basis of density.

Implement BFS and DFS and state what each computes.

Classify DFS edges and use them to detect cycles.

Compute topological order and strongly connected components.

Recognise when a problem is a graph problem in disguise.

1 Representation

A graph is vertices and edges, and the first engineering decision is how to store them. The answer depends almost entirely on density.

Operation	Adjacency list	Adjacency matrix
Space	$\Theta(V + E)$	$\Theta(V^2)$
Test whether (u,v) is an edge	$\Theta(\deg u)$	$\Theta(1)$
Enumerate u 's neighbours	$\Theta(\deg u)$ — optimal	$\Theta(V)$ — you scan a whole row
Add an edge	$\Theta(1)$	$\Theta(1)$
Remove an edge	$\Theta(\deg u)$	$\Theta(1)$

ASK WHICH REGIME YOU ARE IN

Sparse: $E = \Theta(V)$. Road networks, 3D meshes, package dependency graphs, social graphs, the web. Here $O(V + E)$ really is linear, and the adjacency matrix would waste enormous space.

Dense: $E = \Theta(V^2)$. Distance matrices, complete graphs, all-pairs problems. Here $O(V + E)$ is quadratic, and a matrix may be both simpler and faster.

Quoting ' $O(V + E)$ ' without saying which regime applies is how people convince themselves a quadratic algorithm is linear.

Almost all real graphs are sparse, and the adjacency list is the default. A practical refinement for performance-critical code is **compressed sparse row**: two flat arrays, one of neighbour lists concatenated and one of start offsets. It has the same asymptotics and far better cache behaviour than a list of lists.

2 One algorithm, two containers

```
def traverse(G, s, container):
    seen = {s}
    container.add(s)
    while container:
        u = container.remove()      # the only difference
        for v in G[u]:
            if v not in seen:
                seen.add(v)
                container.add(v)

# queue (FIFO) -> BFS;  stack (LIFO) -> DFS
```

Both visit every reachable vertex once and examine every incident edge once, so both are $O(V + E)$. Swapping the container changes the order of exploration, the shape of the resulting tree, and the properties you can extract — which is a surprising amount of consequence for a one-line change.

2.1 BFS

A queue explores all vertices at distance 1, then all at distance 2, and so on. This gives the defining property: **BFS computes shortest paths in unweighted graphs**, because a vertex is first reached along a path of minimum edge count.

- Shortest paths (unweighted), and the distance to every reachable vertex.
- Connected components, by restarting from each unvisited vertex.
- Bipartiteness: two-colour by BFS level; an edge within a level means an odd cycle, so not bipartite.
- Memory is proportional to the *frontier width*, which on a wide shallow graph can approach V .

2.2 DFS

A stack follows one path as far as it goes before backtracking. The interesting structure comes from recording **discovery** and **finish** times for each vertex, which lets every edge be classified.

Edge type	Points to	Meaning
Tree	An undiscovered vertex	Part of the DFS forest.
Back	An ancestor still on the stack	A cycle. A directed graph has a cycle if and only if DFS finds a back edge.
Forward	An already-finished descendant	A shortcut down the tree. Cannot occur in undirected graphs.
Cross	A finished vertex in another subtree	Between branches. Cannot occur in undirected graphs.

CYCLE DETECTION IN FULL

A directed graph contains a cycle **if and only if** a DFS from any starting set discovers a back edge.

($\Rightarrow$) A back edge $u \rightarrow v$ means v is an ancestor of u , so the tree path from v to u plus that edge forms a cycle.

($\Leftarrow$) If a cycle exists, consider the first of its vertices to be discovered; every other vertex on the cycle is reachable from it along undiscovered vertices, so all become its descendants, and the cycle edge returning to it is a back edge.

Memorise the equivalence. It is the basis of deadlock detection, build cycle errors, and recursive type checking.

3 Topological sort

A linear ordering of vertices in which every edge points forward. It exists precisely when the graph is acyclic — a cycle has no consistent ordering.

3.1 Two methods

- **DFS.** Output vertices in decreasing order of finish time. A vertex finishes only after every vertex it points to has finished, so a later finish time means an earlier position in the order.
- **Kahn's algorithm.** Repeatedly output any vertex with in-degree 0 and remove it. If vertices remain but none has in-degree 0, the remainder contains a cycle — which makes cycle reporting natural, and is why build systems tend to use this form.

Applications are everywhere: build systems, task schedulers, spreadsheet recalculation order, course prerequisite chains, instruction scheduling, shader and render-pass ordering. Any time something must happen before something else, you have a DAG and this is the algorithm.

4 Strongly connected components

In a directed graph, u and v are **strongly connected** if each reaches the other. This is an equivalence relation, so it partitions the vertices into **strongly connected components**.

THE CONDENSATION IS ALWAYS A DAG

Contract each SCC to a single node. The resulting graph is guaranteed acyclic — if it had a cycle, all the components on that cycle would be mutually reachable and would therefore have been one component.

This is genuinely useful: it means *any* directed graph can be decomposed into a DAG of strongly connected pieces, and then topologically sorted at the component level. Dataflow analysis and module dependency resolution both rely on exactly this.

4.1 Kosaraju

- 1 Run DFS on G , pushing each vertex onto a stack as it finishes.
- 2 Reverse every edge to form G^T .
- 3 Pop vertices from the stack; each DFS in G^T from an unvisited popped vertex yields exactly one SCC.

Two passes, $O(V + E)$, and about twenty lines of code. Tarjan's algorithm achieves the same in a single pass using low-link values and is faster in practice, at the cost of being harder to remember. Learn Kosaraju for understanding; use Tarjan when it matters.

Applications: deadlock detection, 2-SAT (satisfiable iff no variable shares an SCC with its negation), finding circular module dependencies, and loop identification in compiler dataflow analysis.

5 The table to remember

Question	Algorithm	Cost
Shortest path in an unweighted graph	BFS	$O(V + E)$
Connected components	BFS or DFS from each unvisited vertex	$O(V + E)$
Is the graph bipartite?	BFS, two-colour by level	$O(V + E)$
Does a directed graph have a cycle?	DFS, detect a back edge	$O(V + E)$
Valid execution order	Topological sort (DFS or Kahn)	$O(V + E)$
Mutually reachable groups	Kosaraju or Tarjan	$O(V + E)$
Single points of failure	DFS with low-link (articulation points, bridges)	$O(V + E)$

WHAT IS ACTUALLY WORTH LEARNING HERE

Every algorithm in this module is linear and short. You will rarely implement them from scratch — they are in every standard library and every graph package.

The durable skill is **recognition**: noticing that build ordering, deadlock, type inference, dependency resolution, mesh connectivity, render-graph scheduling, state machines, and puzzle solving are all graph search in disguise.

Once you see the graph, the algorithm is a table lookup. Getting to the graph is the work.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.006 — Graphs, BFS, DFS, Topological Sort	Careful development of both traversals and the properties each yields.
Jeff Erickson, Algorithms — Depth-First Search chapter	The best free treatment of edge classification and SCCs, with proofs that are actually readable.
Competitive Programmer's Handbook — Graph chapters	Implementation-focused, with the practical details (CSR layout, iterative DFS) that theory courses omit.
CSES Problem Set — Graph Algorithms section	Around 35 auto-graded problems covering exactly this module. The best free practice available.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a graph with both adjacency list and adjacency matrix. Measure neighbour iteration and edge-existence queries on a sparse and a dense graph, and confirm the predicted crossover.
- 2 Implement BFS and DFS from the single traverse skeleton, differing only in the container. Verify both visit the same vertex set.
- 3 Use BFS to test bipartiteness. Construct a graph with an odd cycle and confirm detection.
- 4 Implement DFS with discovery and finish times and classify every edge. Verify on a directed graph that back edges appear exactly when a cycle exists.
- 5 Implement topological sort both ways — reverse DFS finish order and Kahn's algorithm. Make the Kahn version report the cycle when one exists.
- 6 Implement Kosaraju's algorithm. Verify that contracting the components yields a DAG by running your cycle detector on the condensation.
- 7 Convert a recursive DFS to an explicit-stack iterative version and demonstrate that it survives a path graph of 10^6 vertices where the recursive one overflows.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 When would you choose an adjacency matrix, and what does $E = \Theta(V)$ versus $\Theta(V^2)$ change about the complexity $O(V + E)$?
- 2 BFS and DFS differ in one line. What is it, and what consequences follow?
- 3 What exactly does BFS compute that DFS does not, and why?
- 4 State and prove both directions of the back-edge / cycle equivalence.
- 5 Why does reverse DFS finish order produce a valid topological order?
- 6 Why is the condensation of a directed graph always a DAG, and why is that useful?